

CSE 331: Computer Security Fundamentals

Spring 2026

Basic Cryptography:
Symmetric Encryption, Modes of Operation

Amir Rahmati
Xinrui Yu

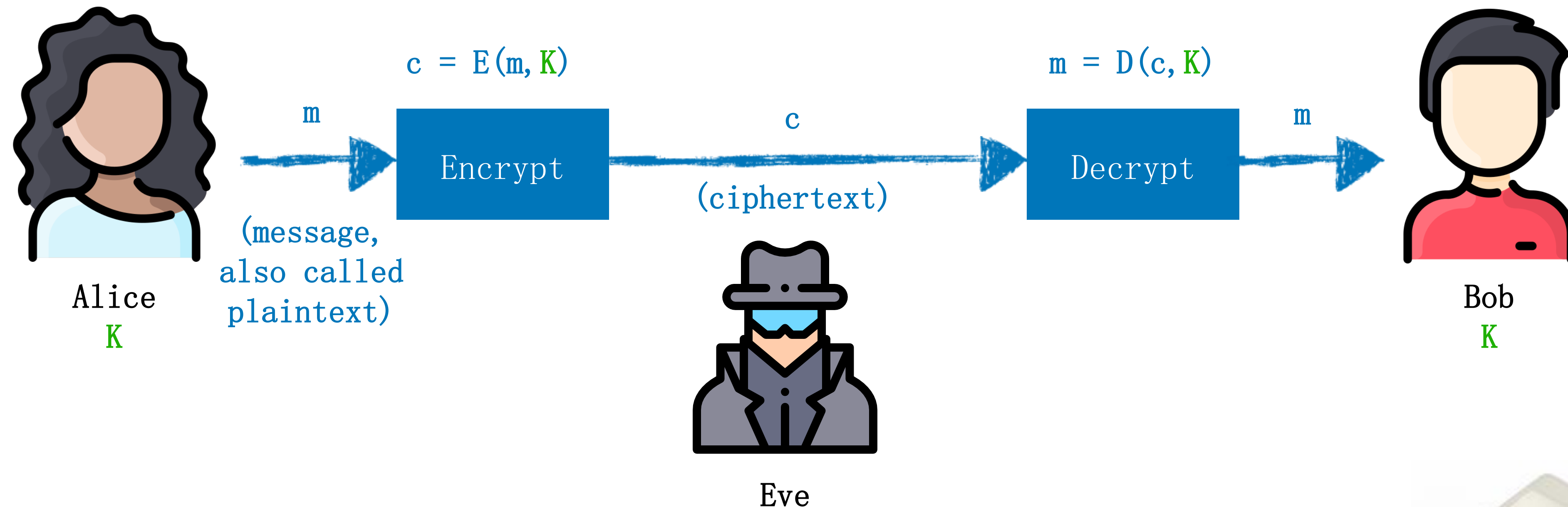


Stony Brook Institute
at Anhui University

安徽大学纽约石溪学院

Symmetric Setting

Both communicating parties have access to a **shared random string K** , called the **key**.



Eavesdropper knows c and E/D , tries to find m

Brute force/brute forcing: trying all possible keys



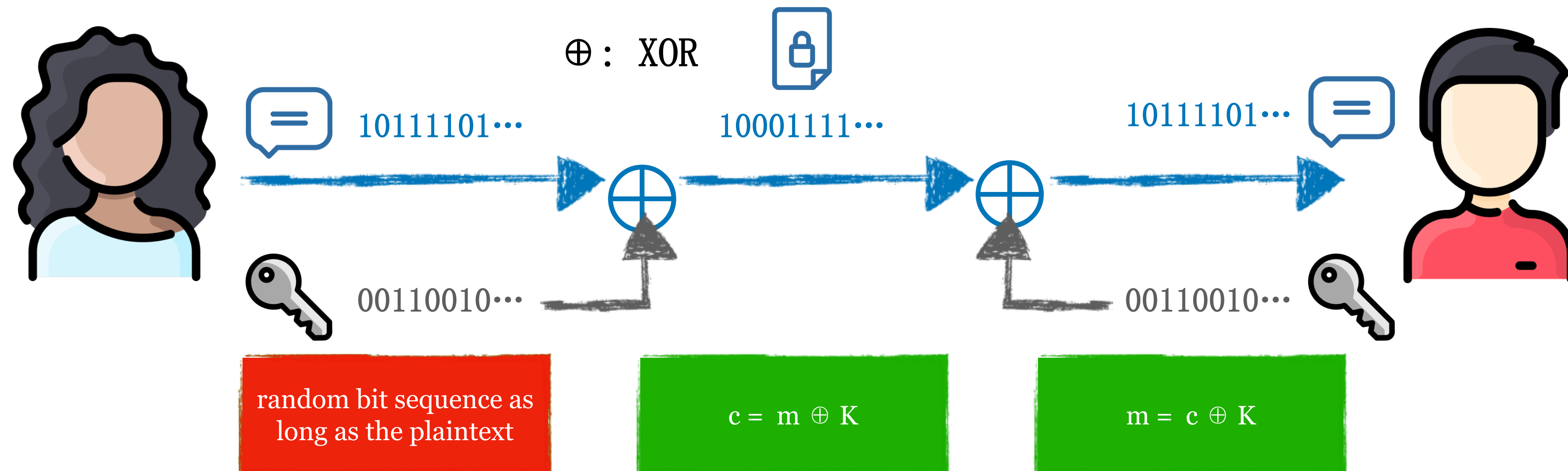
Symmetric Setting

Is it possible to break all ciphers
with brute forcing?

No.

(We will show you why)

One-Time Pad



- (Claude Shannon, 1949) Cipher achieves **perfect confidentiality** if and only if:
 - there are as many possible keys as possible plaintexts.
 - every key is equally likely.
- **Can you brute force?**

Important Concept: Key Space

Key space is the number of possible keys for an encryption scheme.
For a given ciphertext, every unique key maps to a possible plaintext.

Example (Assume the length of the ciphertext is n):

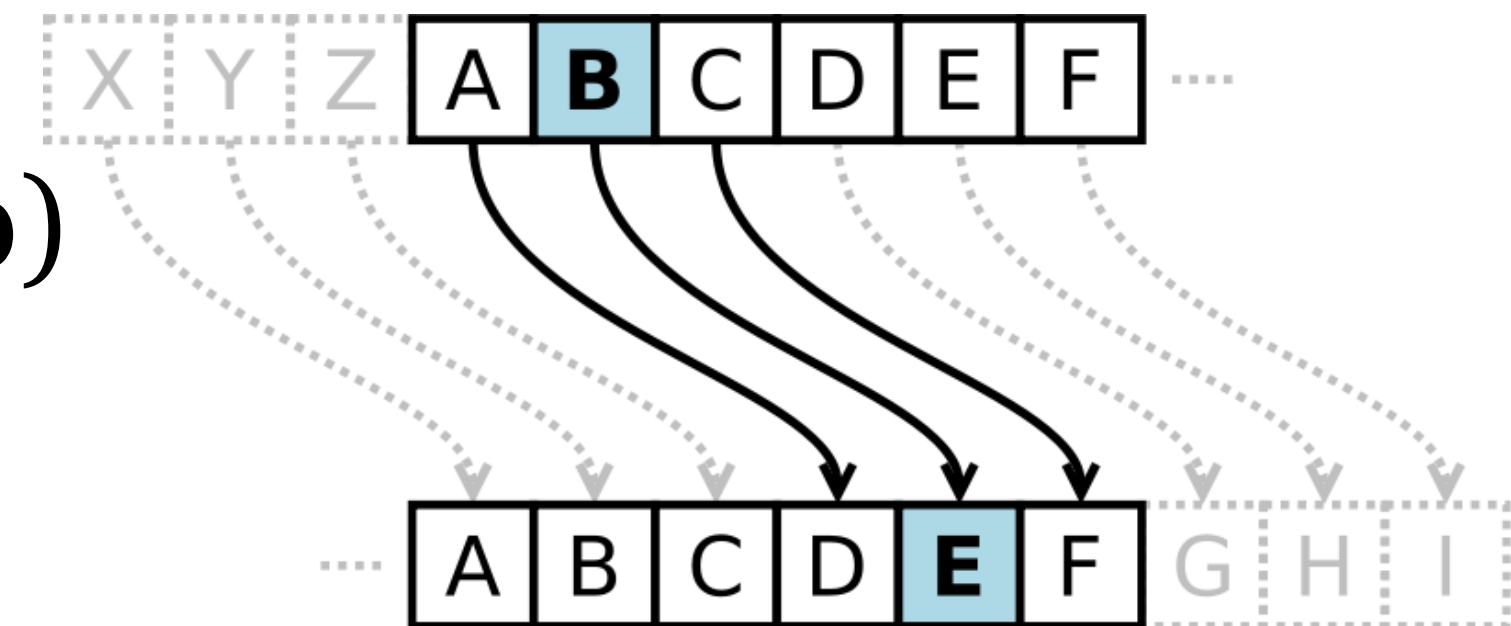
Cesar Cipher (Shift Cipher) 26 (or 25, excluding 0)

Simple substitution cipher 26!
(every letter is replaced with a different one)

ABCDEFGHIJKLMNOPQRSTUVWXYZ

ZEBRAS C D F G H I J K L M N O P Q T U V W X Y

One-Time Pad 2^n



- Modern cryptography mostly assumes that the ciphertext is sufficiently long.

Important Concept: Key Space

Simple substitution cipher, ciphertext length is n

Key space: $26!$

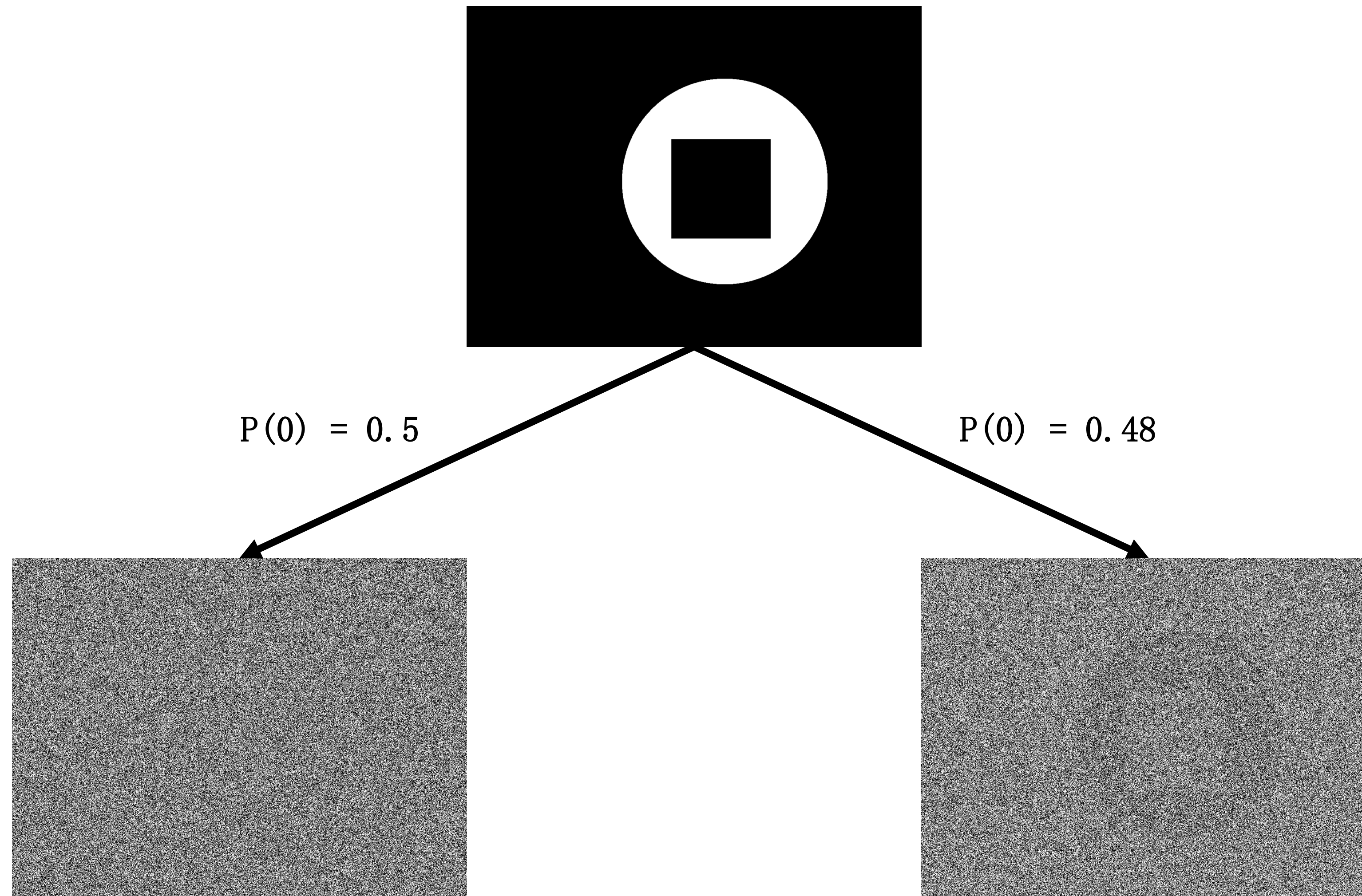
Plaintext space or message space: 26^n Very few possible plaintexts are meaningful.

$$\lim_{n \rightarrow \infty} \frac{K}{P}?$$

Key space becomes extremely small compared to plaintext space, so any meaningful words from brute forcing indicates correct key.

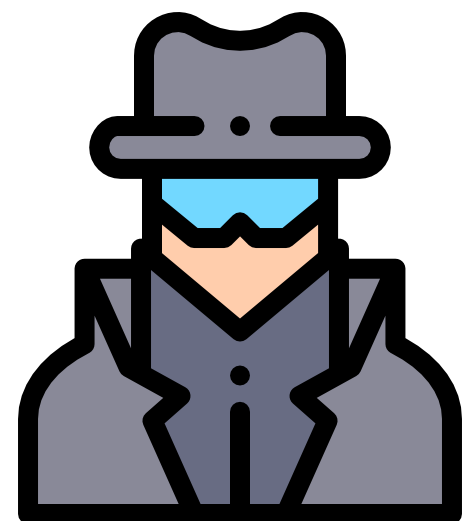
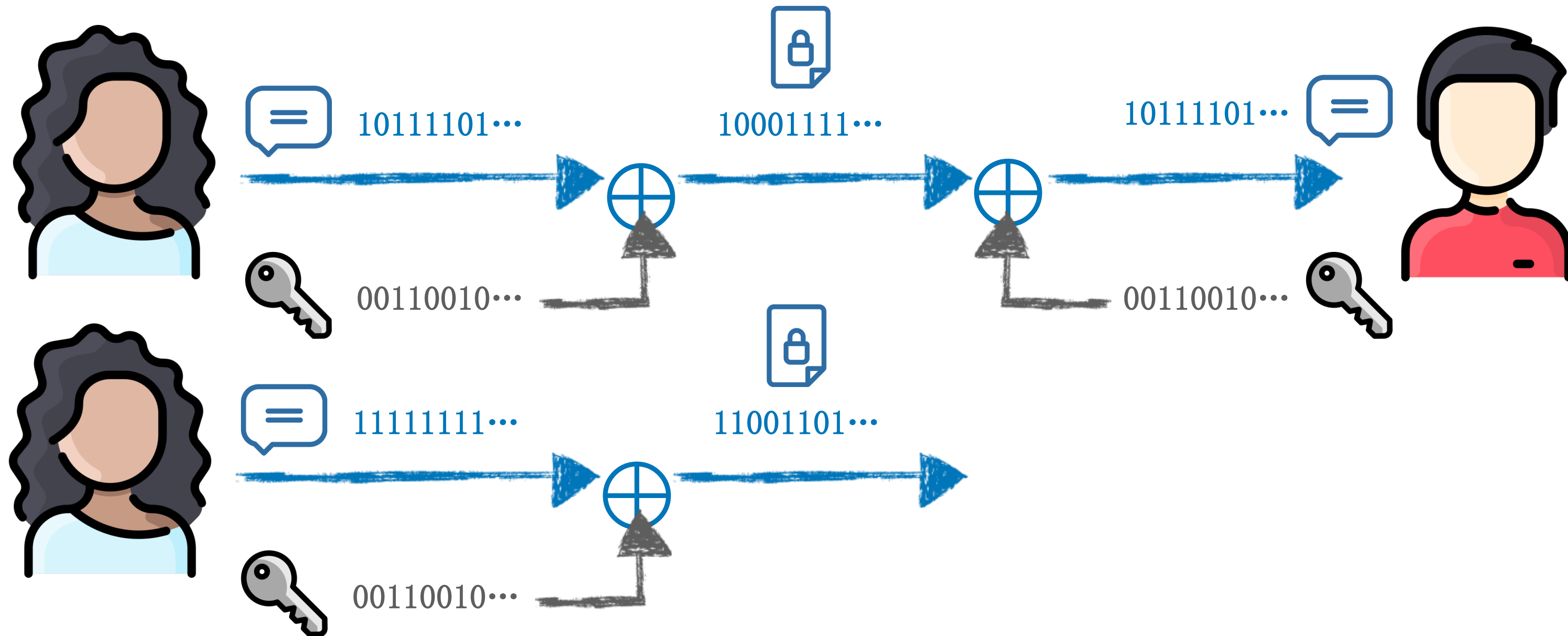
One-Time Pad has the same key space and plaintext space. You never know if the result indicates correct key -> impossible to brute force.

When the key is not uniformly distributed (distribution bias)



Key Reuse

(when the key is shorter than plaintext)



The attacker can learn relationship between plaintexts

$$C1 \oplus C2 = (P1 \oplus K) \oplus (P2 \oplus K) = (P1 \oplus P2) \oplus (K \oplus K) = P1 \oplus P2$$

The attacker can learn relationship between plaintexts. What happens?



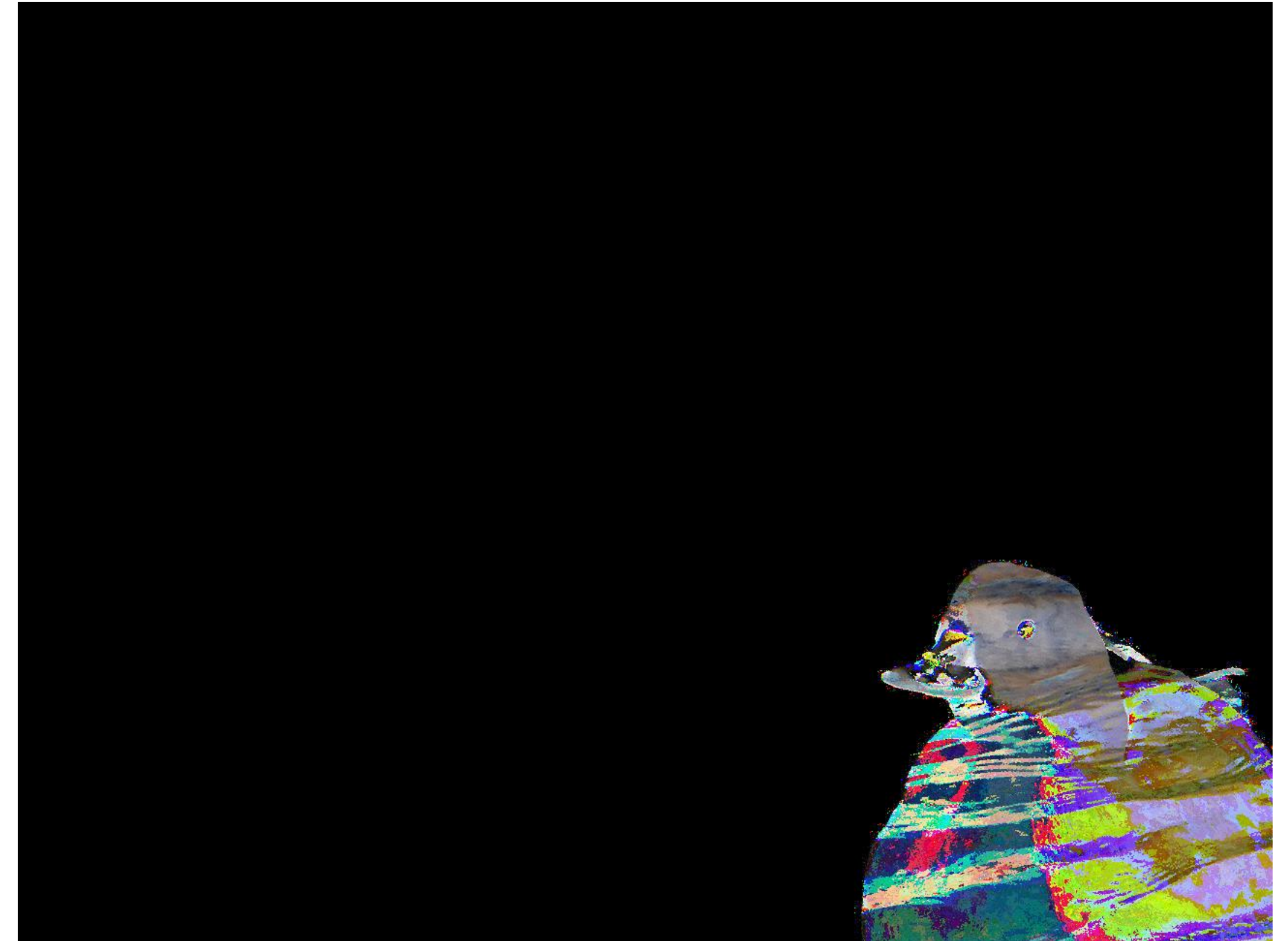
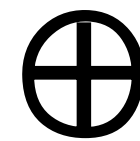
$\oplus$



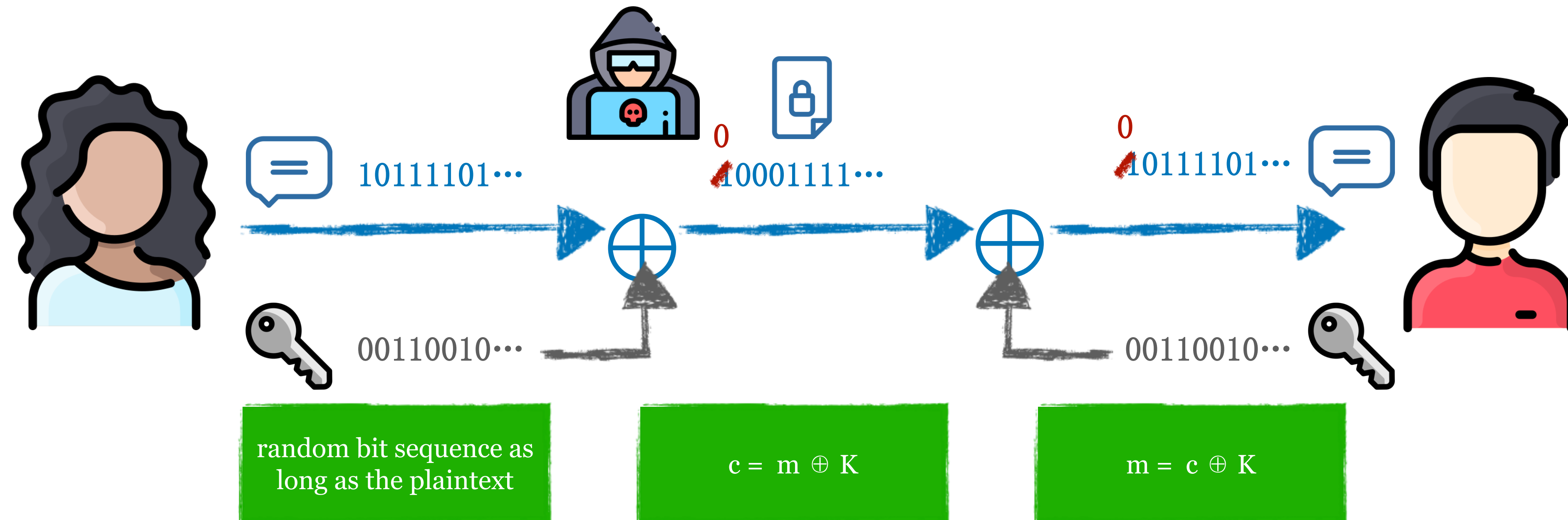
The Secret
Number is
123456789



The attacker can learn relationship between plaintexts. What happens?



Integrity?



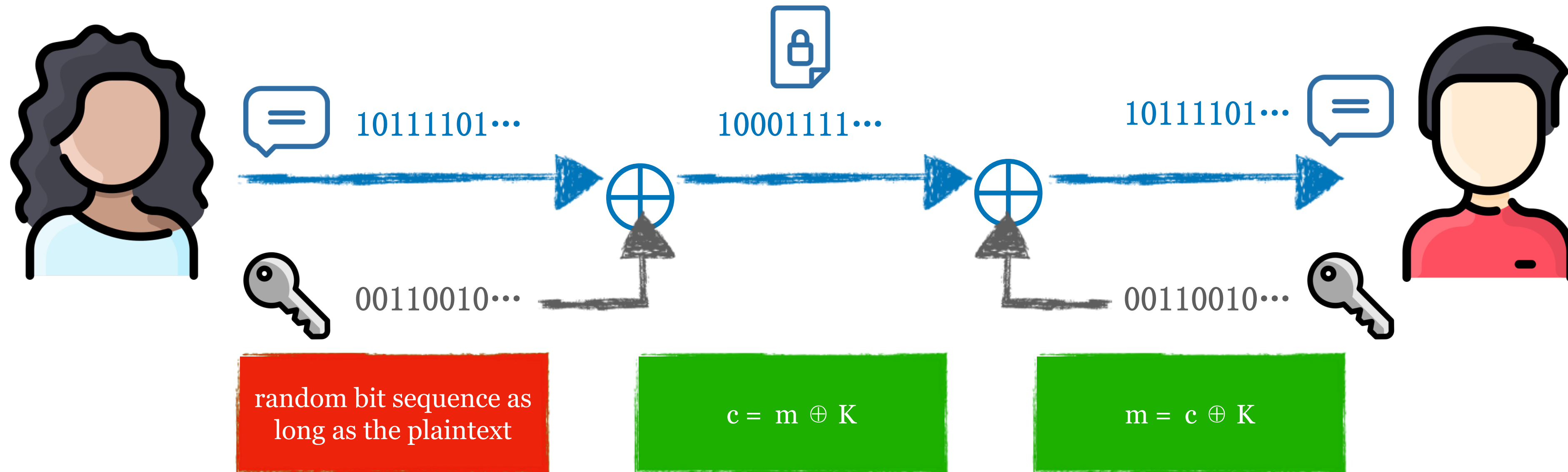
Advantages of One-Time Pad

- Easy to compute:
 - Encryption and decryption are the **same operation**.
 - Bitwise XOR is **very cheap to compute**.
- As secure as theoretically possible:
 - **Given a ciphertext, all plaintexts are equally likely, regardless of attacker's computational resources**
 - ...as long as the key sequence is truly random.
 - **True randomness is expensive to obtain in large quantities**
 - ...as long as each key is same length as plaintext.
 - **But how does sender communicate the key to receiver?**

Disadvantages of One-Time Pad

- **Key must be as long as the plaintext**
 - Impractical in most realistic scenarios
 - Still used for diplomatic and intelligence traffic
- **Insecure if keys are reused**
 - Attacker can obtain XOR of plaintexts
- **Does not guarantee integrity**
 - One-time pad only guarantees confidentiality
 - Attacker cannot recover plaintext, but can easily change it to something else

One-Time Pad



Solution

Reduce the
required key size!

But how?

Use special cryptographic primitives:
block ciphers, stream ciphers

- + Single key can be re-used (with some restrictions).
- Not as theoretically secure as one-time pad.

Stream Cipher vs. Block Cipher

- **Stream Cipher:** encrypts data one bit or byte at a time. Key space increases with the length of the plaintext.
- **Block Cipher:** encrypts data in fixed-size blocks. Key space is fixed.

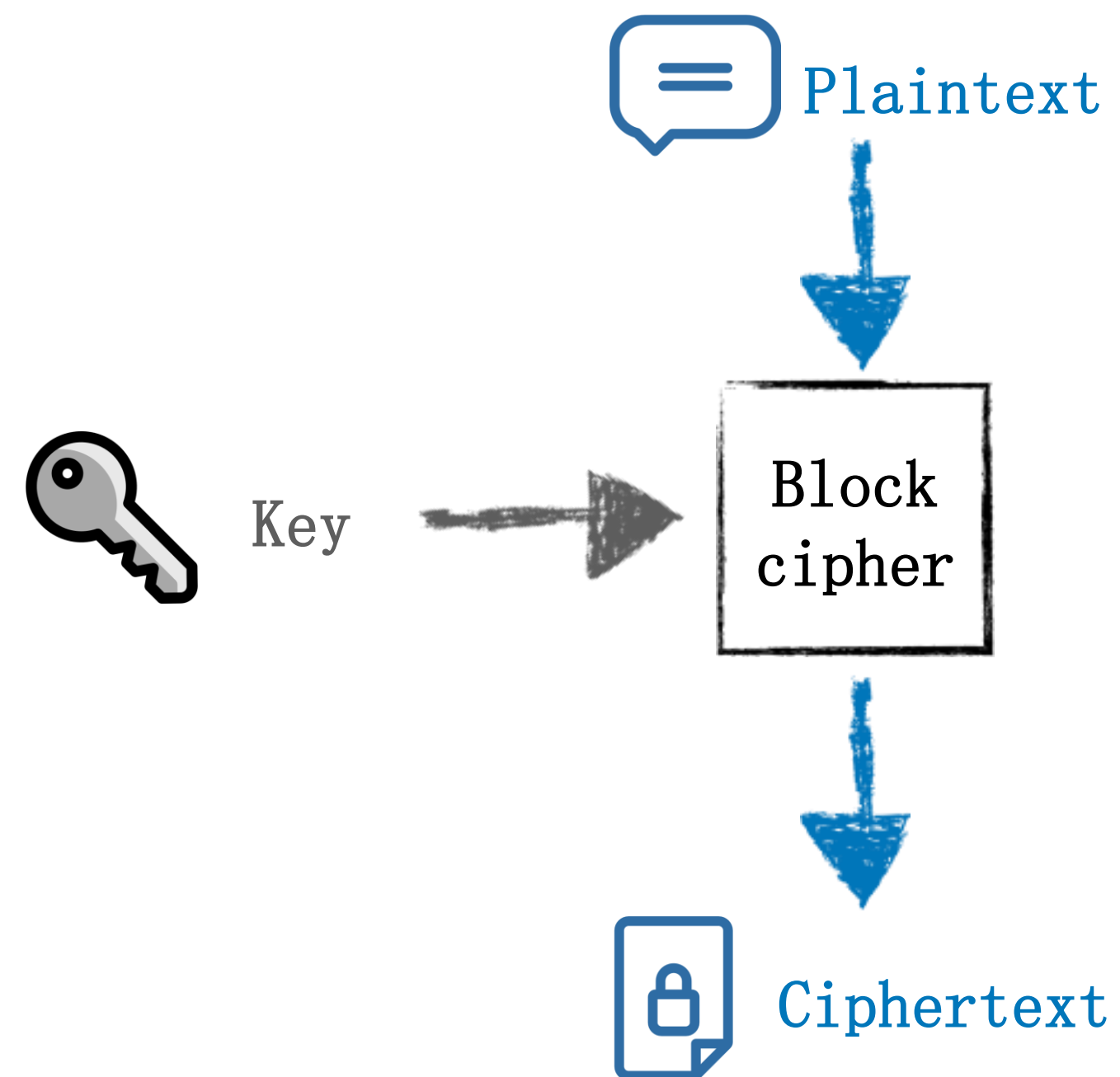
Stream Ciphers

- One-time pad: $\text{Ciphertext}(\text{Key}, \text{Message}) = \text{Message} \oplus \text{Key}$
 - Key must be a random bit sequence as long as message
- Idea: replace “random” with “pseudo-random”
 - Use a pseudo-random number generator (PRNG)
 - PRNG takes a short, truly random secret seed and expands it into a long “random-looking” sequence
 - E.g., 128-bit seed into a 10^6 -bit pseudo-random sequence
- $\text{Ciphertext}(\text{Key}, \text{Msg}) = \text{Msg} \oplus \text{PRNG}(\text{Key})$
 - Message processed bit by bit (like one-time pad)

No efficient algorithm can tell this sequence from a truly random one

Block Ciphers

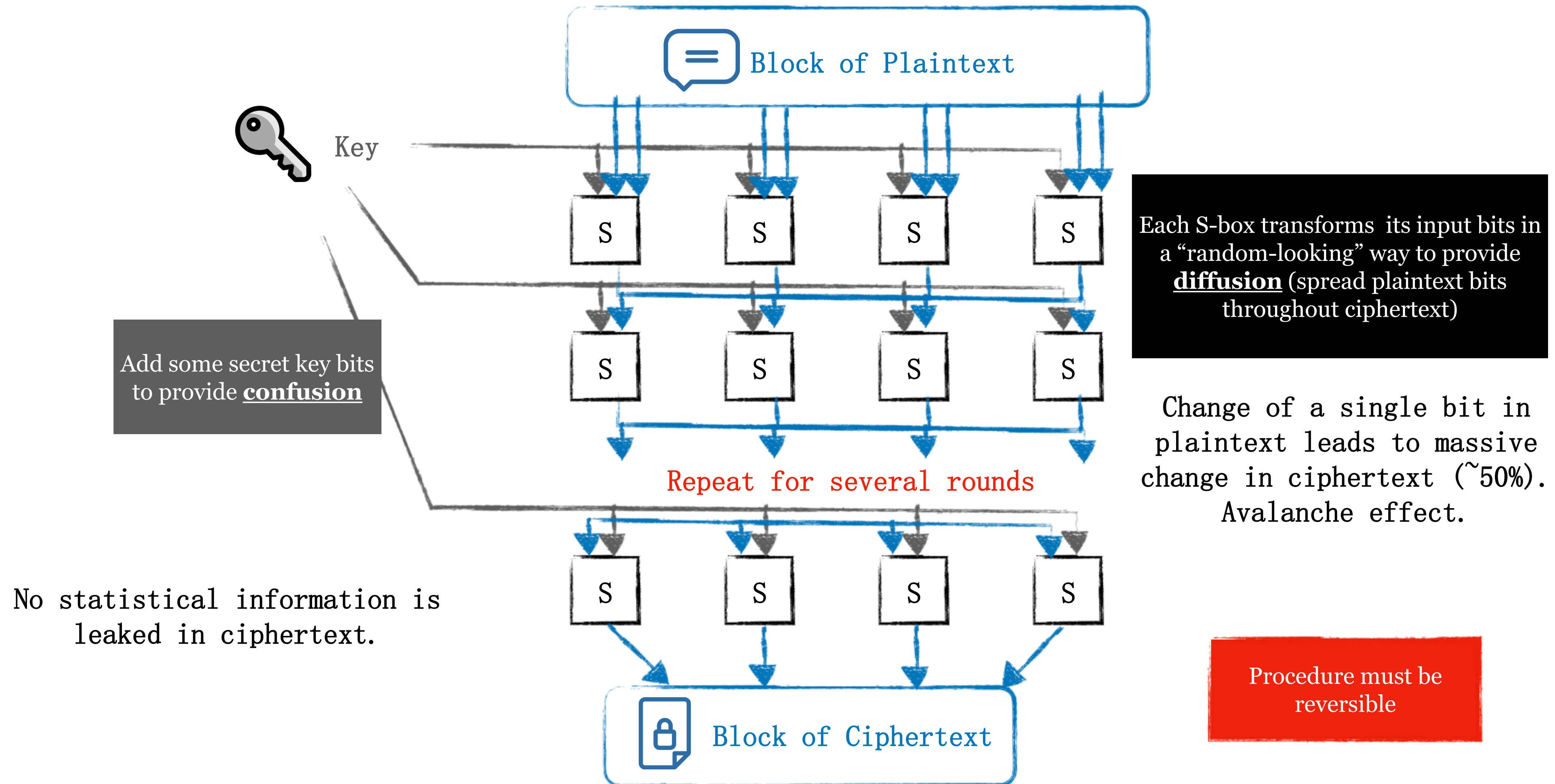
- Operates on a single chunk (“block”) of plaintext
 - For example, 64 bits for DES, 128 bits for AES
 - Each key defines a different **permutation**
 - Same key is reused for each block (can use short keys)



Block Cipher Security

- Result should look like a **random permutation on the inputs**
 - Recall: not just shuffling bits. N-bit block cipher permutes over 2^N inputs.
- Only computational guarantee of secrecy
 - **Not impossible to break, just very expensive**
 - If there is no efficient algorithm (unproven assumption!), then can only break by brute-force, try-every-possible-key search
 - **Time and cost of breaking the cipher exceed the value and/or useful lifetime of protected information**

Block Cipher Operation (Simplified)



Standard Block Ciphers

- **DES: Data Encryption Standard**
 - Feistel structure: builds invertible function using non-invertible ones
 - Invented by IBM, issued as federal standard in 1977
 - 64-bit blocks, 56-bit key + 8 bits for parity

Data Encryption Standard

- 56 bit keys are quite short

Key Size (bits)	Number of Alternative Keys	Time required at 1 encryption/ μ s	Time required at 10^6 encryptions/ μ s
32	$2^{32} = 4.3 \times 10^9$	$2^{31} \mu\text{s} = 35.8 \text{ minutes}$	2.15 milliseconds
56	$2^{56} = 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1142 \text{ years}$	10.01 hours
128	$2^{128} = 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.4 \times 10^{24} \text{ years}$	$5.4 \times 10^{18} \text{ years}$
168	$2^{168} = 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.9 \times 10^{36} \text{ years}$	$5.9 \times 10^{30} \text{ years}$
26 characters (permutation)	$26! = 4 \times 10^{26}$	$2 \times 10^{26} \mu\text{s} = 6.4 \times 10^{12} \text{ years}$	$6.4 \times 10^6 \text{ years}$

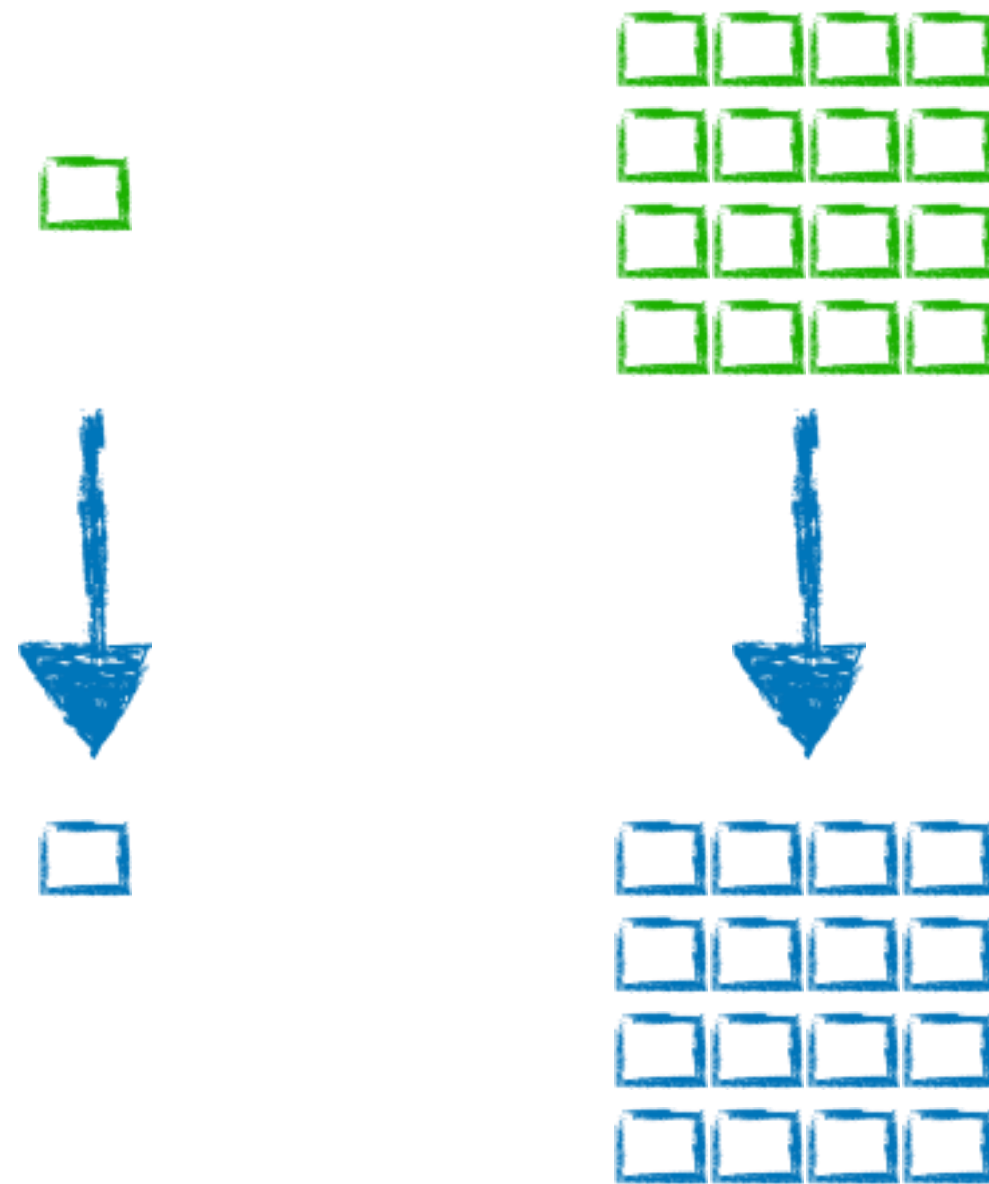
- < 24 hours to find DES key
- DES ---> 3DES
 - 3DES: DES + inverse DES + DES (with 2 or 3 diff keys)

Standard Block Ciphers

- **DES: Data Encryption Standard**
 - Feistel structure: builds invertible function using non-invertible ones
 - Invented by IBM, issued as federal standard in 1977
 - 64-bit blocks, 56-bit key + 8 bits for parity
- **AES: Advanced Encryption Standard**
 - New federal standard as of 2001
 - NIST: National Institute of Standards & Technology
 - Based on the Rijndael algorithm
 - Selected via an open process
 - 128-bit blocks, keys can be 128, 192 or 256 bits (10,12,14 rounds)

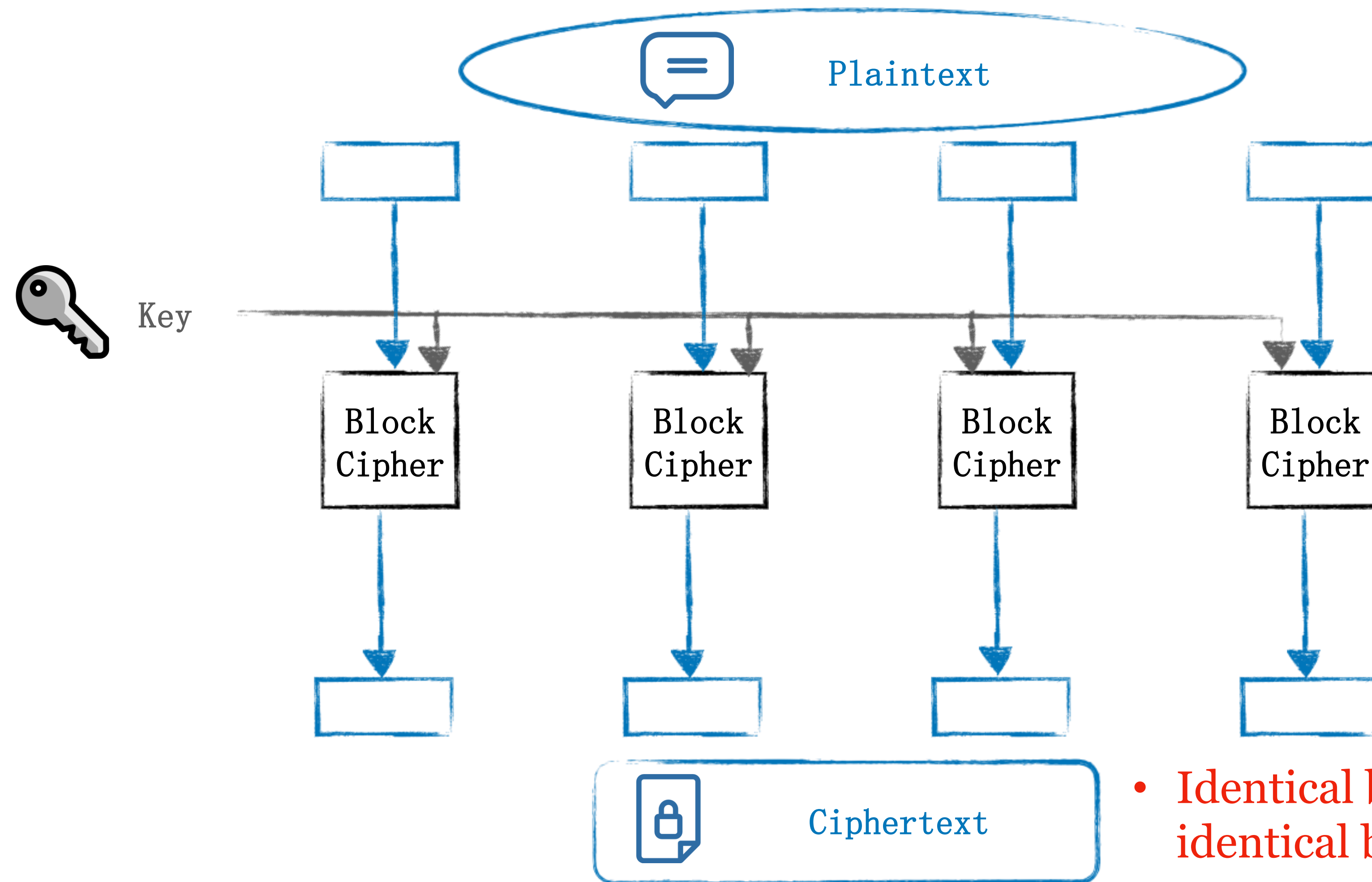
Encrypting a Large Message

So, we've got a good block cipher, but our plaintext is larger than 128-bit block size



What should we do?

Electronic Code Book (ECB) Mode

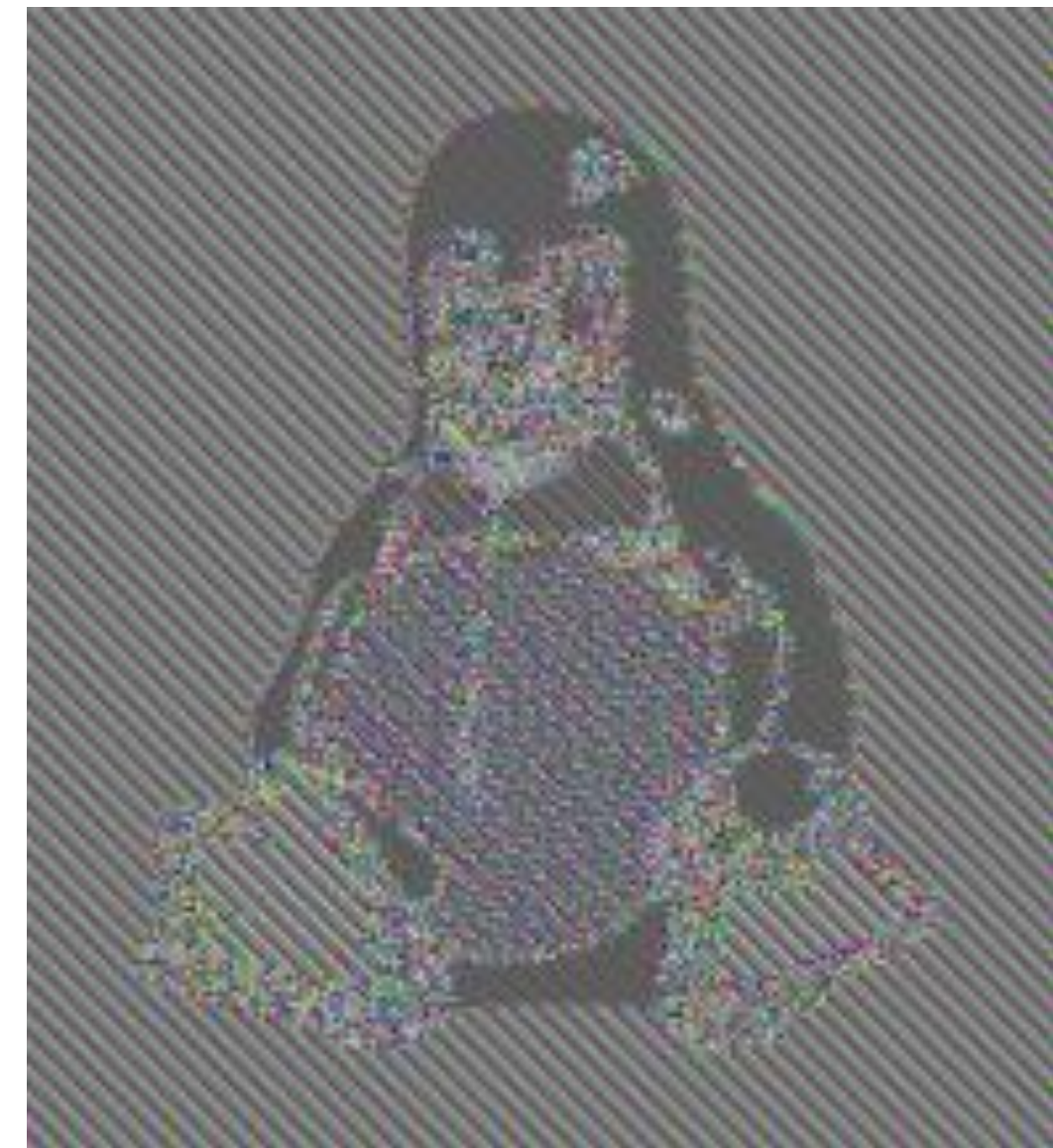


- Identical blocks of plaintext produce identical blocks of ciphertext
- No integrity checks: can mix and match blocks

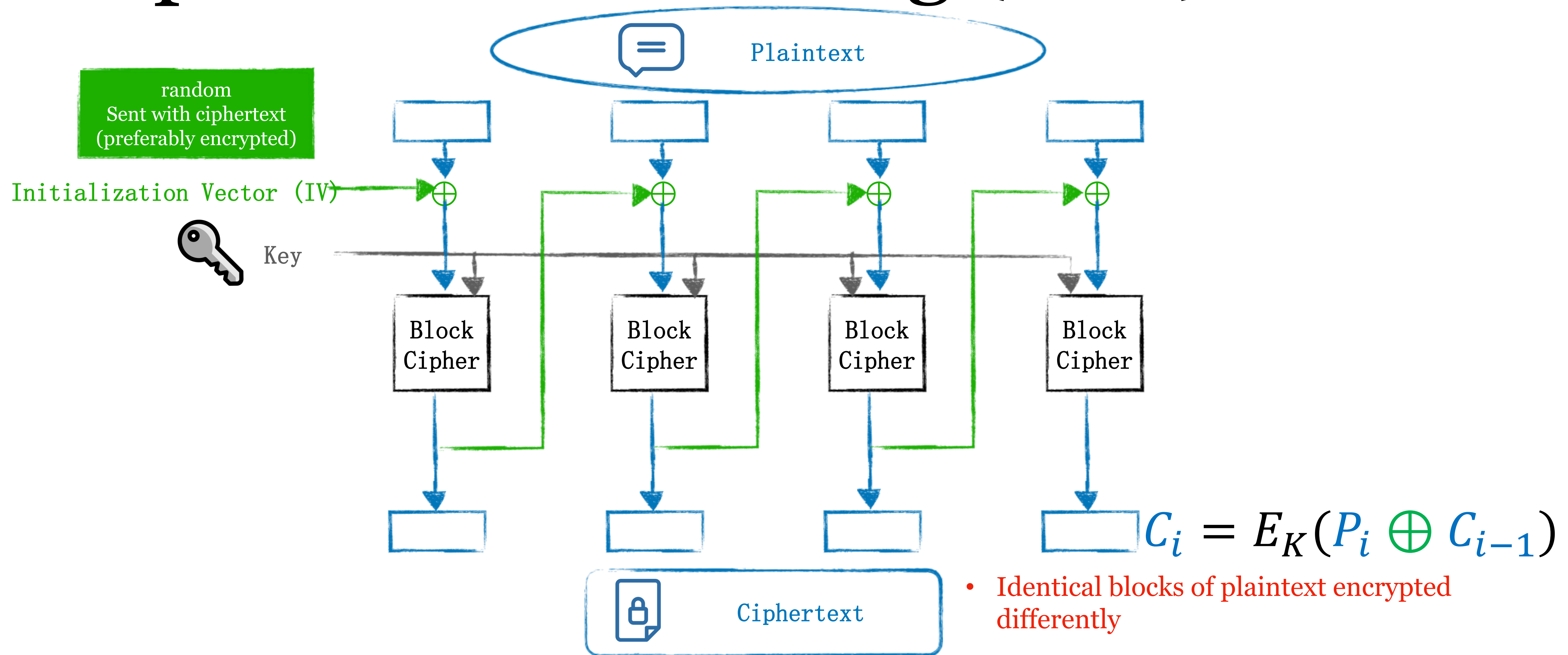
Information Leakage in ECB Mode



Encrypt in ECB mode



Cipher Block Chaining (CBC) Mode

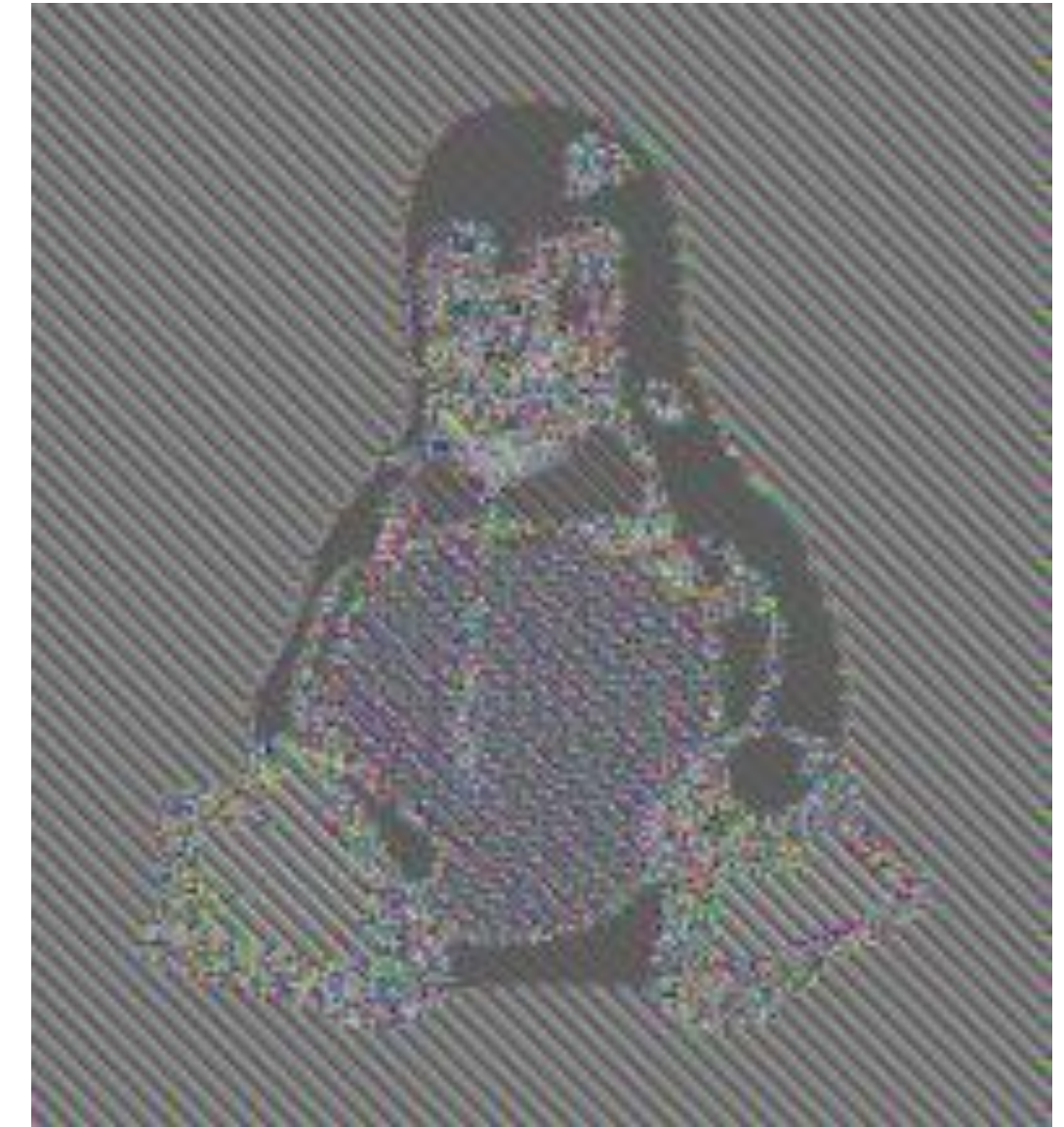


- Identical blocks of plaintext encrypted differently
- Last cipherblock depends on entire plaintext
- Still does not guarantee integrity

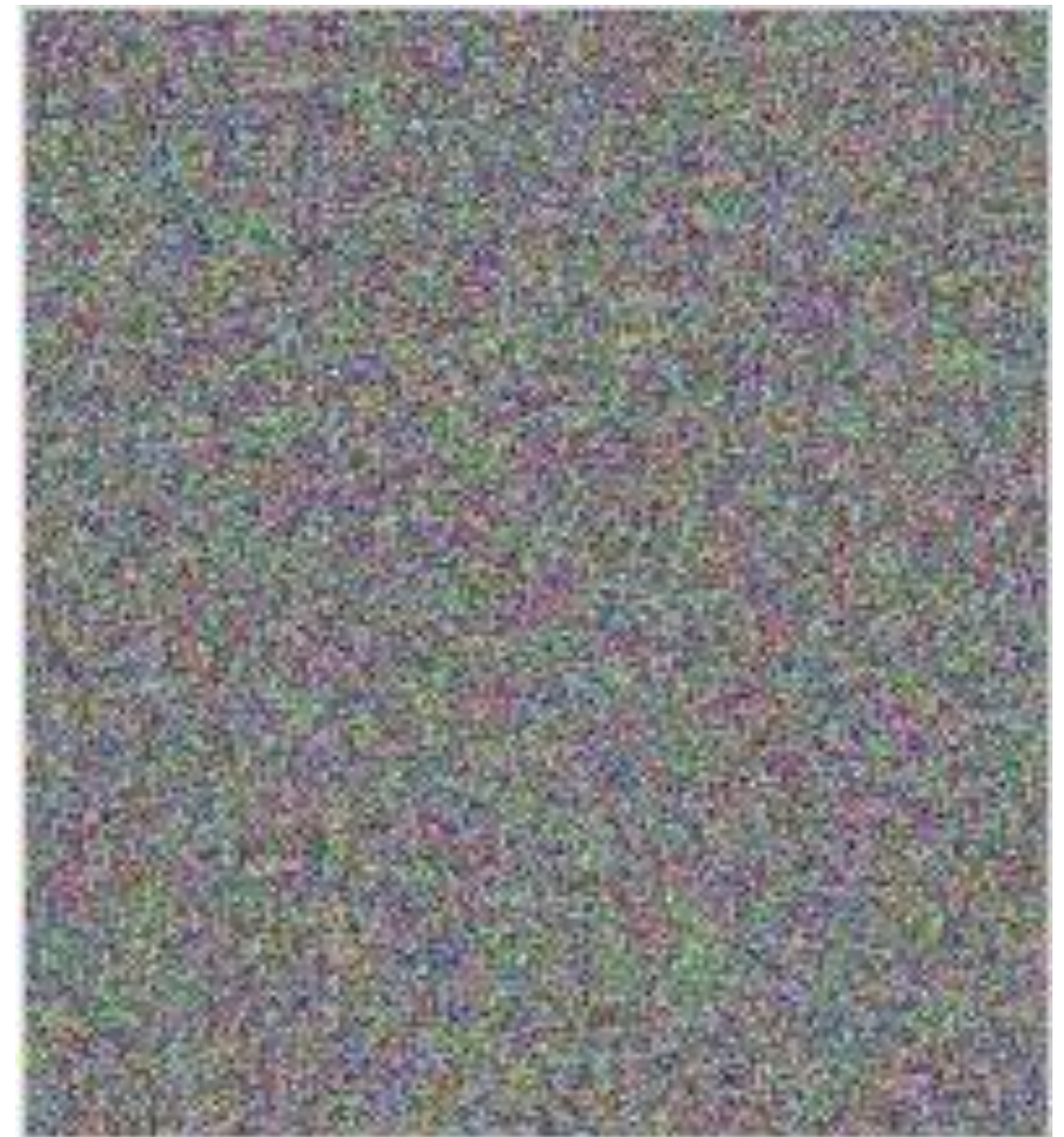
ECB vs. CBC



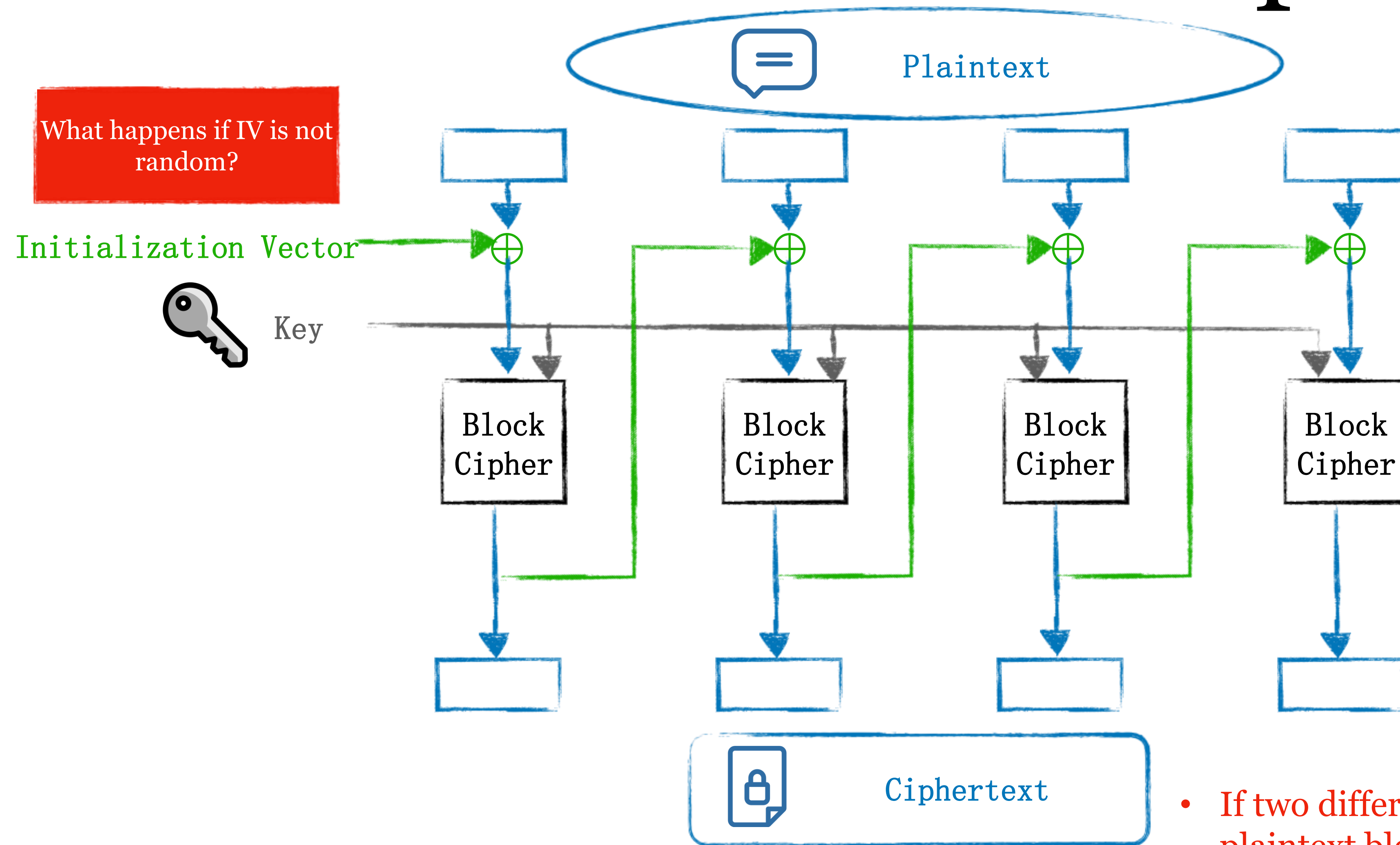
Encrypt in ECB mode



Encrypt in CBC mode



CBC: When IV Repeats

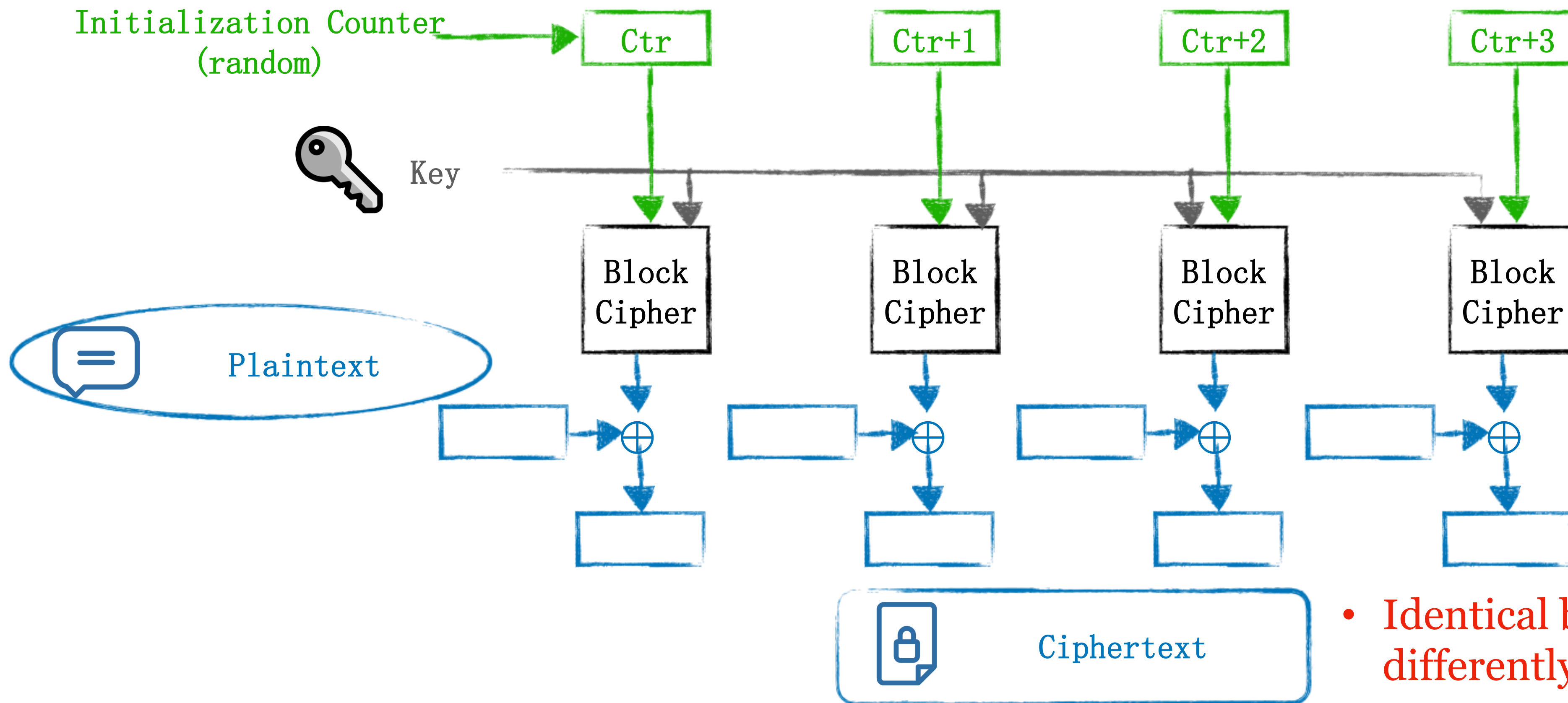


- If two different messages start with the same plaintext block, their encryptions will start with the same ciphertext blocks.

Number Used Once (nonce)

- Assign a number to a message. Typically starts at zero, for example.
- Construct a nonce using the message number.
 - If sending data in two directions, add a direction indicator.
 - With a specific key, nonce used only once.
 - Nonce size = block size.
- Encrypt nonce with block cipher = IV
 - Add enough information inside ciphertext so that the receiver can re-compute nonce, and therefore IV

Counter (CTR) Mode

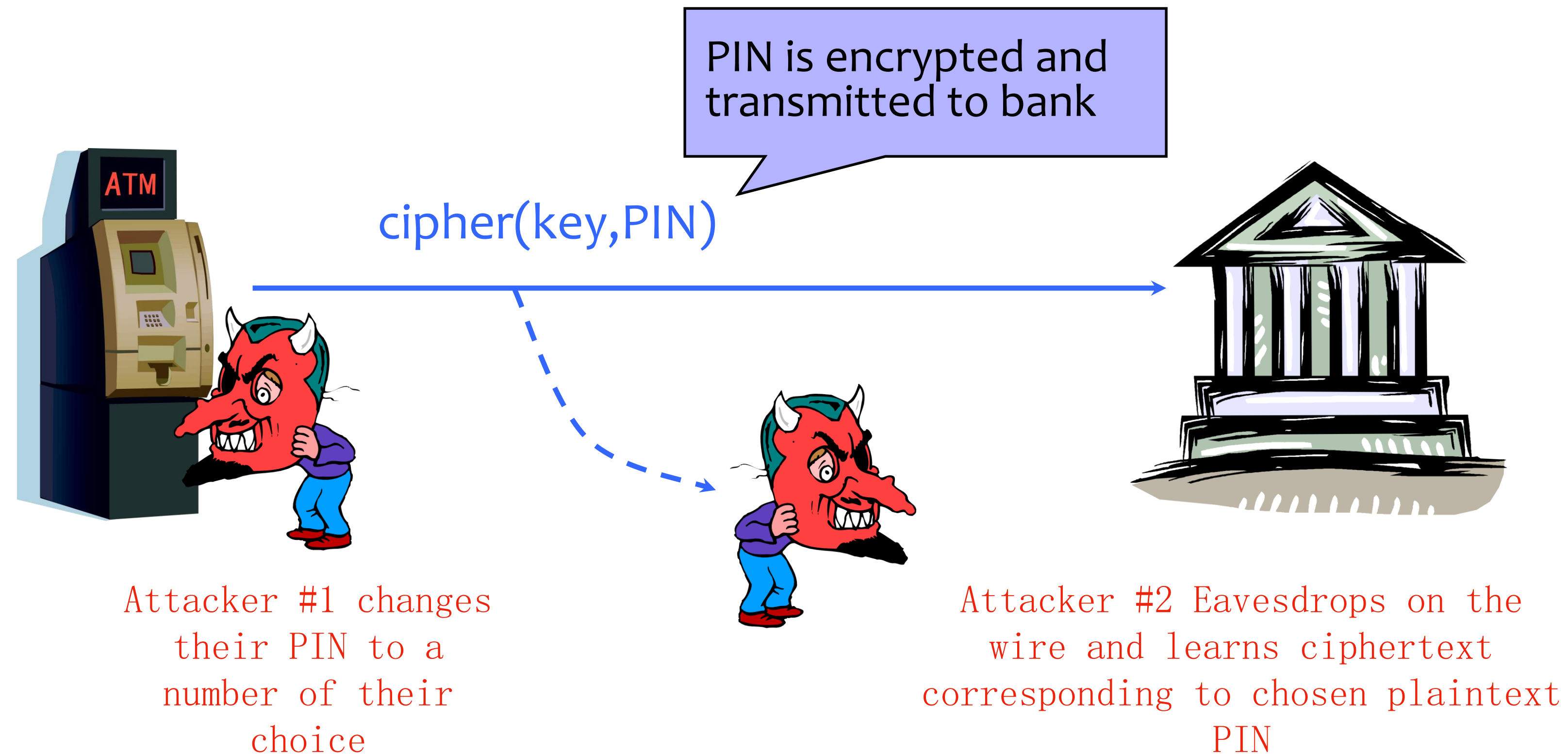


- Identical blocks of plaintext encrypted differently
- Still does not guarantee integrity; Fragile if ctr repeats

How Can a Cipher Be Attacked?

- Attacker knows ciphertext and encryption algorithm
 - What else does the attacker know?
 - Depends on the application in which the cipher is used!
- Ciphertext-only attack
- KPA: Known-plaintext attack (stronger)
 - Knows some plaintext-ciphertext pairs
- CPA: Chosen-plaintext attack (even stronger)
 - Can obtain ciphertext for any plaintext of his choice
- CCA: Chosen-ciphertext attack (very strong)
 - Can obtain ciphertext for any plaintext of his choice,
 - **and** decrypt any ciphertext except the target

Chosen Plaintext Attack



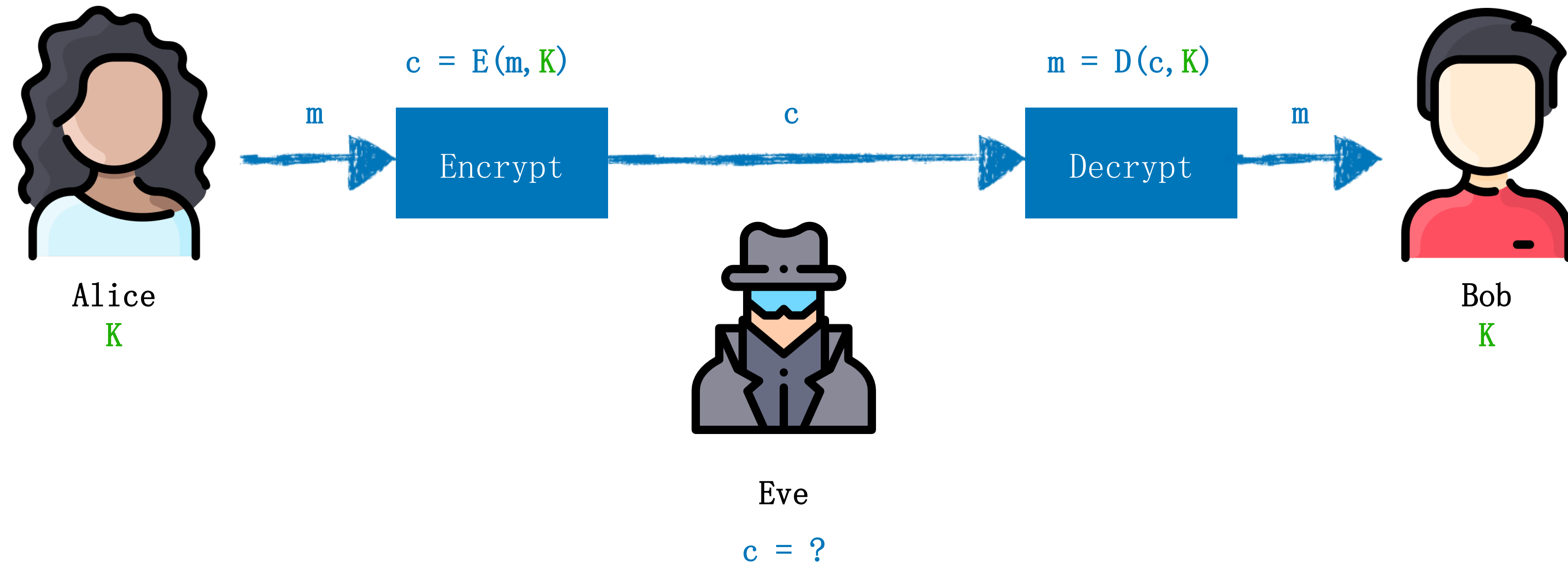
... repeat for any PIN value

Very Informal Intuition

Minimum security
requirement for a
modern encryption
scheme

- **Security against chosen-plaintext attack (CPA)**
 - Ciphertext leaks no information about the plaintext
 - Even if the attacker correctly guesses the plaintext, he cannot verify his guess
 - Every ciphertext is unique, encrypting same message twice produces completely different ciphertexts
 - Implication: encryption must be randomized or stateful
- **Security against chosen-ciphertext attack (CCA)**
 - Integrity protection
 - it is not possible to change the plaintext by modifying the ciphertext

So Far: Achieving Confidentiality



To Achieve Integrity

Message Authentication Schemes (MAC)

